

Stack (Restricted form of an array).

like an array:-

- ① also is a ADS
- ② elements are homogenous
- ③ also has index.

A linear ADS is a DS in which after one element we have just one more element

diff. than array -

- (1) open from one end only.
- (2) Does not allow random access only sequential access. (LIFO)

Important terminology for stack.

- (1) Push $()$: It is a method / process for storing data in a stack.
- (2) Pop $()$: It is a method for deleting the data from stack. It always deleted the top element.
- (3) (Peek) $()$: It is a method for accessing top element without deleting it.
- (4) TOS: It is called top of stack and it is an integer which holds the current index in the stack.
- (5) Stack overflow - It is an error message which is generated when an attempt is made to store data in already full / occupied stack.
- (6) Stack underflow - Empty data stack में peek or pop करने से त्रुटि

Peek :-

- ① Underflow
- ② Stack underflow & return
- ③ return the element pointed by Tos.

Program :-

```

Public class stack {
    Private int arr;
    private int tos;
    private int size;
    public stack (int size) {
        arr = new int [size];
        this.size = size;
        tos = -1;
    }

    Public void push (int x) {
        if (tos == size - 1) {
            SOP ("stack overflow");
            return;
        }
        tos++;
        arr[tos] = x;
    }

    public int pop () {
        if (tos == -1) {
            SOP ("Stack underflow");
            return -1;
        }
        int x = arr[tos];
        tos--;
        return x;
    }
}
    
```

```

public int peek () {
    if (tos == -1) {
        SOP ("stack underflow");
        return -1;
    }
    int x = arr[tos];
    return x;
}
} return arr[tos];

```

```

public class use_stack {
    PSVM (String args[])
    Scanner kb = new Scanner (System.in);
    SOP ("Enter size of stack");
    int size = kb.nextInt();
    Stack s = new Stack (size);

```

~~if~~

```

for (int i=1; i<=size+1; i++) {
    SOP ("Enter element to push");
    int x = kb.nextInt();
    s.push(x);
    SOP ("Element " + x + " pushed successfully");
}
for (int i=1; i<=size+1; i++) {
    int x = s.pop();
    SOP ("element " + x + " popped successfully");
}
}
}

```

Queue

- (1) it is a linear data structure.
- (2) like a array it is also open from both the ends.
- (3) But its both ends are not used for both the operations.
- (4) its Rear (Back) end is used from Insertion and its front end is used for deletion.
- (5) Insertion $\rightarrow$ enqueue() & deletion $\rightarrow$ dequeue();
- (6) Since enqueue() & dequeue take place on opposite ends we say a Queue follows FIFO.

Algorithm for enqueue():-

- (1) Check for overflow:
- (2) if Queue is full then print the Msg Queue Overflow and return.
- (3) if Queue is not full then increment Rear by 1.
- (4) Insert the element in the Queue at the index pointed by Rear.
- (5) Finish & return.

Algorithm for dequeue():-

- (1) Check for underflow:
- (2) if Queue is empty then print the msg Queue underflow and return.
- (3) if Queue is not empty then delete the element pointed by front.
- (4) increment front by 1.
- (5) return the deleted element.

Algorithm for peek:-

- 1 Check for underflow.
- 2 if Queue is Empty then print the Msg Queue underflow and return.
- 3 if Queue is not Empty then copy the element pointed by front.
- 4 return the copied element.

queueDemo.queueImpl;

class
public Queue {

private int [] arr;

private int front, rear;

private int size;

public Queue (int size) {

arr = new int[size];

this.size = size;

rear = -1;

front = 0;

}

public void enqueue (int data) {

if (rear == size - 1) {

throw new QueueException ("Queue Over
Flow");

} rear ++;

arr [rear] = data;

}

public int dequeue () {

if (front > rear) {

throw new QueueException ("Queue
underflow")

```

    }
    int data = arr[front];
    front++;
    return data;
}

```

```

public int peek () {
    if (front > rear) {
        throw new QueueException ("Queue underflow");
    }
    return arr[front];
}

```

```

public class UseQueue {
    public static void main (String[] args) {
        Scanner kb = new Scanner (System.in);
        sop ("Enter size");
        int size = kb.nextInt (); int x, choice;
        Queue queue = new Queue (size);
        do {
            sop ("Select an operation");
            sop ("1. Enqueue \n 2. Dequeue \n 3. Peek \n 4. Quit");
            int choice = kb.nextInt ();
            switch (choice) {
                case 1:
                    try {
                        sop ("Enter element to enqueue");
                        x = kb.nextInt ();
                        queue.enqueue (x);
                        sop ("Element inserted");
                    }

```

```
catch (QueueException ex) {
    sop (ex.getMessage());
    break;
}
```

Case 2:

```
- try {
    x = queue.dequeue();
    sop ("Element deleted ! : " + x);
}
```

```
catch (QueueException ex) {
    sop (ex.getMessage());
    break;
}
```

Case 3: ~~for~~

```
- try {
    x = queue.peek();
    sop ("Top Element ! : " + x);
}
```

```
catch (QueueException ex) {
    sop (ex.getMessage());
    break;
}
```

Case 4:

```
sop ("Thank you for using the app !");
break;
```

default :

```
sop ("wrong choice");
}
while (choice != 4);
}
```

Implement

(i) using array

fixed

(ii) Arraylist

growable

(iii) Linked List

Dynamic

Algorithm:-

Push

- (1.) Check for Overflow
- (2.) If stack is full then display the message stack overflow and return.
- (3.) Increment TOS by 1.
- (4.) Increase the Element in the Stack at the position pointed by TOS.
- (5.) return

POP

- (1) Check the Underflow
- (2) display msg stack underflow and return
- (3) delete the Element pointed by TOS from stack.
- (4) Decrement by 1.
- (5) Return the deleted Element.